

OPNSENSE

FIREWALL RULES LAB

ICMP Filtering, Aliases & Rule-Order Evaluation

Technical Lab Report

Prepared for: Cybersecurity Lab Exercise (Lab 01/02)
Environment: OPNsense Firewall (VirtualBox) + Kali Linux Client
Date: June 30 – July 1, 2026

Index

This index lists every section of the report in the order the work was performed, from planning through to the final verified ruleset.

1. Lab Objective	3
1.1 Lab Environment	3
2. Step 1 — Creating a Firewall Alias	4
3. Step 2 — Creating the Block Rule	5
4. Step 3 — Verifying the Rule with Ping Tests	6
4.1 Test 1 — Pinging 8.8.8.8 (single host in alias)	6
4.2 Test 2 — Pinging 8.8.4.4 and 8.8.8.8 (full alias)	6
4.3 Results Summary	7
5. Step 4 — Understanding Rule Order (First-Match Evaluation)	8
5.1 Misconfiguration — Allow rule placed above the Block rule	8
5.2 Correction — Restoring proper rule order	9
5.3 Rule-order takeaway	10
6. Step 5 — Supplementary Rule: Blocking a Specific Host	10
7. Final Verified Ruleset	11
8. Answers to Exercise Questions	12
9. How Firewall Rule Order Affects Traffic Filtering	13
10. Conclusion	14

1. Lab Objective

This report documents a hands-on exercise performed on an OPNsense firewall deployed inside Oracle VirtualBox, with a Kali Linux virtual machine acting as the LAN client. The objective of the lab was to configure and test custom firewall rules on the LAN interface, group destination hosts using a firewall alias, verify rule behaviour with live connectivity tests, and observe how the order of rules changes the outcome of traffic filtering.

The exercise covers the complete workflow an administrator follows in a real deployment: planning the rule, building a reusable alias, writing the rule, applying the configuration, validating the result from the client machine, and finally correcting a rule-ordering mistake that caused the filter to behave unexpectedly.

Scope of this report

Firewall alias creation, ICMP block rule on the LAN interface, ping-based verification, rule-order (first-match) troubleshooting, and a supplementary host-based block rule.

1.1 Lab Environment

Component	Details
Hypervisor	Oracle VirtualBox
Firewall	OPNsense (live media), management IP 192.168.10.1
Client VM	Kali Linux – Firefox browser + terminal
Interfaces configured	WAN and LAN
Access method	OPNsense Web GUI (root@OPNsense.internal)
Primary tools used	Firewall › Aliases, Firewall › Rules › LAN, ping (ICMP)

2. Step 1 — Creating a Firewall Alias

Before writing the rule, a reusable Firewall Alias named GoogleDNS was created under Firewall › Aliases. An alias groups one or more hosts, networks, or ports under a single named object so that rules can reference the group instead of individual IP addresses — this makes the ruleset easier to read and to maintain.

Configuration

Type: Host(s). Content: 8.8.8.8 and 8.8.4.4 (Google's public DNS resolvers). Name: GoogleDNS. Purpose: groups Google's DNS servers so a single rule can filter traffic to both addresses at once.

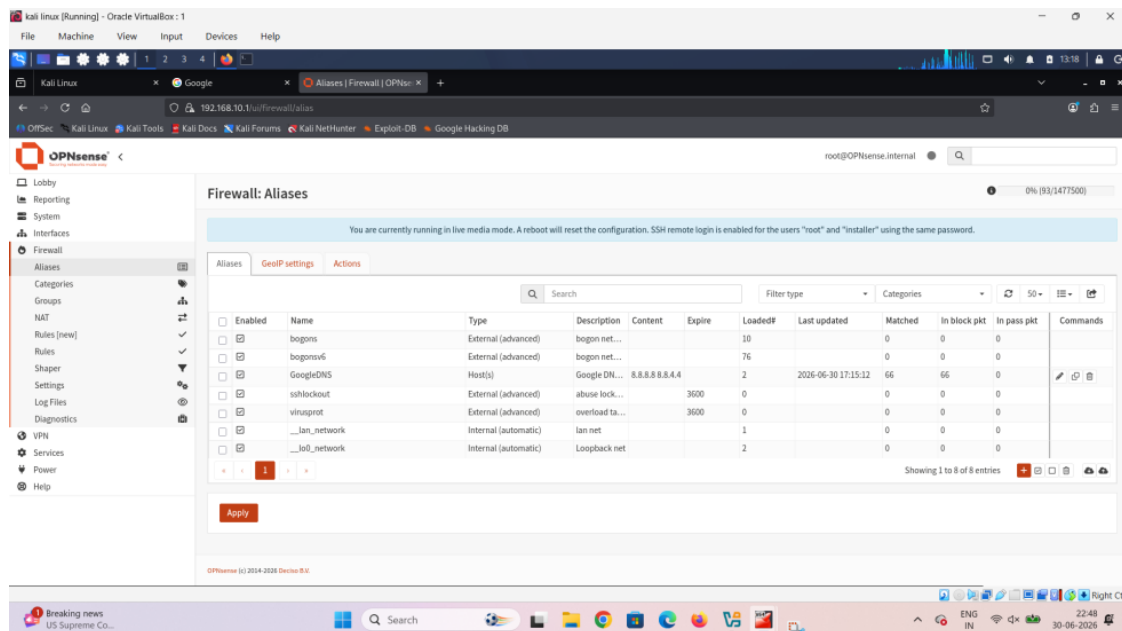


Figure 1: Firewall › Aliases page showing the GoogleDNS alias (Host(s) type, content 8.8.8.8 / 8.8.4.4) alongside the system-default aliases. The Matched and “In block pkt” counters confirm the alias is actively being evaluated by the firewall.

Observation

The alias table shows 66 matched packets and 66 blocked packets against GoogleDNS at the time of the screenshot — direct evidence that the block rule referencing this alias was actively intercepting ICMP traffic during testing.

3. Step 2 — Creating the Block Rule

With the alias in place, a new rule was added under Firewall › Rules › LAN to block ICMP (ping) traffic from the LAN network toward the GoogleDNS alias. On OPNsense/pfSense-based systems, LAN rules are evaluated top-to-bottom on a first-match basis, so placement of this rule relative to the default “allow LAN to any” rule is critical (this is explored further in Section 5).

Rule parameters

Field	Value
Action	Block
Protocol	IPv4 ICMP
Source	LAN net
Destination	GoogleDNS (alias: 8.8.8.8/32, later 8.8.8.8 + 8.8.4.4)
Description	Block ICMP to Google DNS

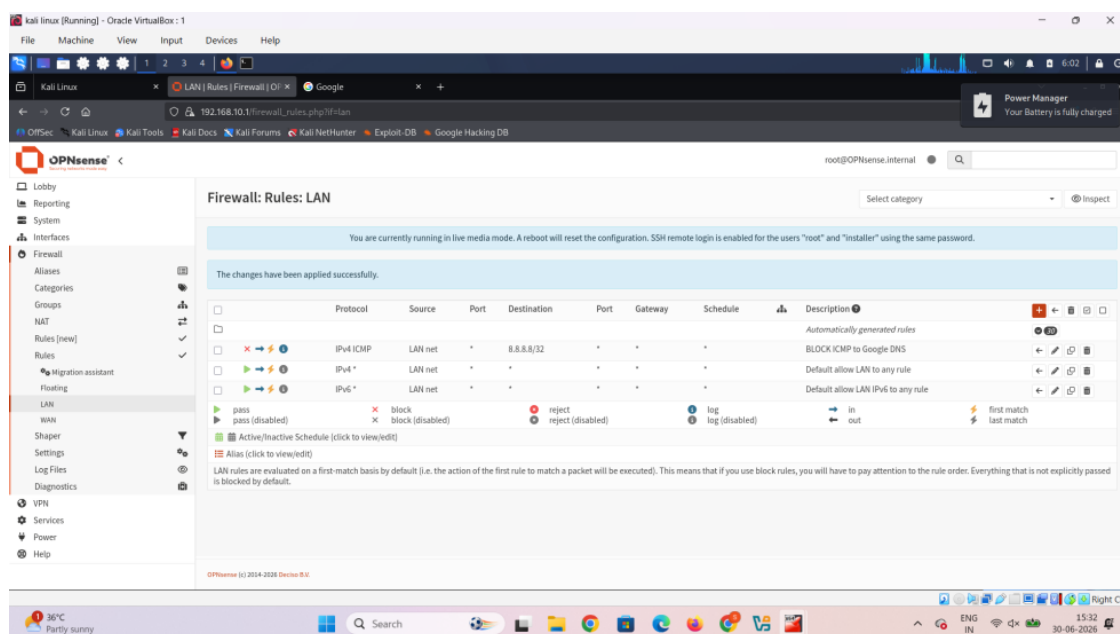


Figure 2: Firewall › Rules › LAN after adding the “Block ICMP to Google DNS” rule. The red X icon marks it as a Block action, positioned above the automatically generated “Default allow LAN to any rule.” The banner confirms the changes were applied successfully.

Placing the block rule above the default allow rule is essential: because LAN rules match on a first-match basis, a block rule placed after an allow-all rule would never be reached, and the traffic would already have been permitted.

4. Step 3 — Verifying the Rule with Ping Tests

To confirm the rule was working as intended, ICMP connectivity was tested from the Kali Linux terminal using the ping command, both before and after the rule (and its later alias expansion) were applied.

4.1 Test 1 — Pinging 8.8.8.8 (single host in alias)

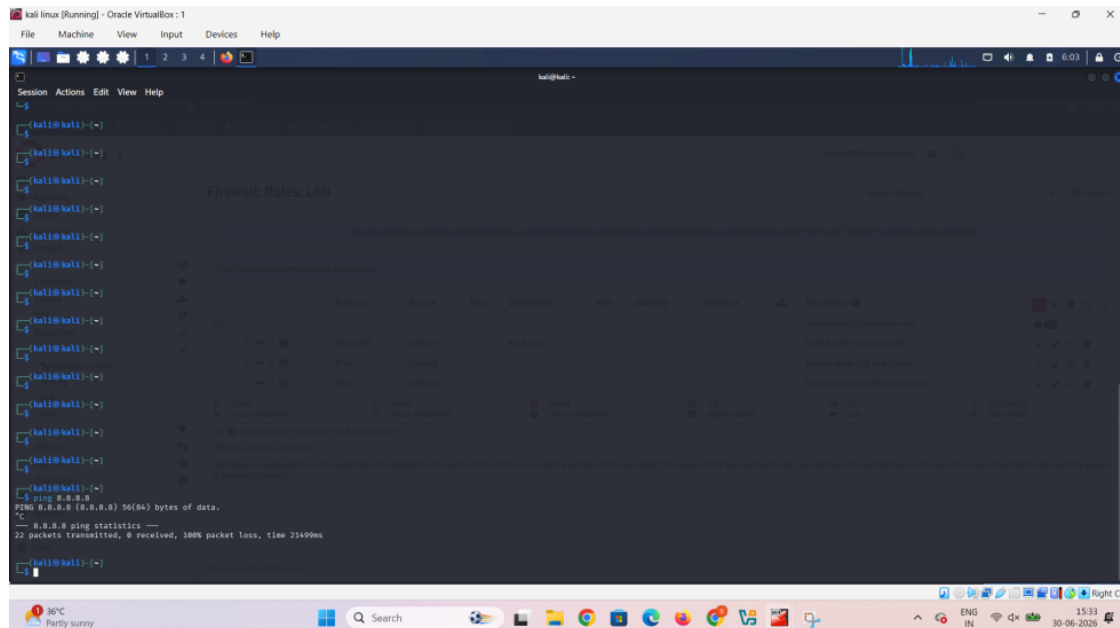


Figure 3: Kali terminal output for ping 8.8.8.8. All 22 transmitted packets resulted in 100% packet loss, confirming that the OPNsense block rule is successfully dropping outbound ICMP requests to Google's primary DNS server.

4.2 Test 2 — Pinging 8.8.4.4 and 8.8.8.8 (full alias)

After expanding the GoogleDNS alias to include both 8.8.8.8 and 8.8.4.4, both addresses were re-tested to confirm the alias update took effect on the existing rule without needing to duplicate the rule itself.

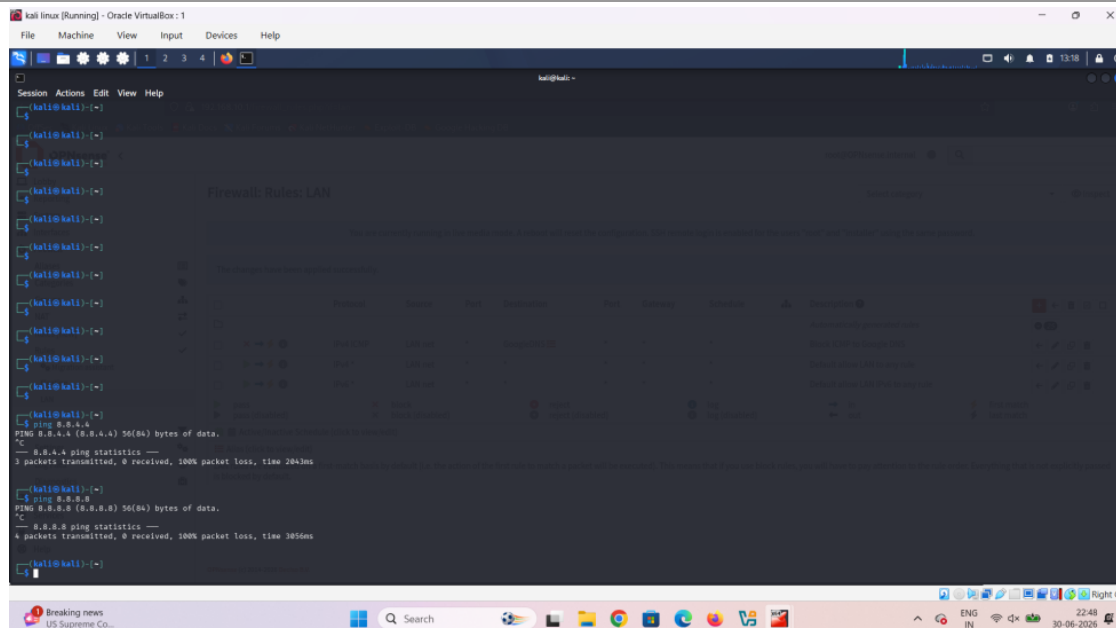


Figure 4: Kali terminal output for ping 8.8.4.4 followed by ping 8.8.8.8. Both tests show 100% packet loss (3 and 4 packets transmitted respectively), confirming the block rule now covers every host defined inside the GoogleDNS alias.

4.3 Results Summary

Target	Packets Sent	Packets Received	Packet Loss	Result
8.8.8.8 (initial rule)	22	0	100%	Blocked — as expected
8.8.4.4 (after alias update)	3	0	100%	Blocked — as expected
8.8.8.8 (after alias update)	4	0	100%	Blocked — as expected

Conclusion of test

100% packet loss on every test confirms the LAN block rule correctly matched and dropped ICMP Echo Requests destined for both members of the GoogleDNS alias, validating the rule and alias configuration end to end.

5. Step 4 — Understanding Rule Order (First-Match Evaluation)

OPNsense evaluates firewall rules on each interface from top to bottom and stops at the first rule that matches a packet (“first match wins”), unless that rule is explicitly marked to continue evaluation. This means the position of a rule in the list is just as important as its content. To demonstrate this, the ruleset was intentionally reordered and the effect on connectivity was observed.

5.1 Misconfiguration — Allow rule placed above the Block rule

A rule named “Pass ALL Traffic” (IPv4 *, LAN net → any) was moved above the “Block ICMP to Google DNS” rule.

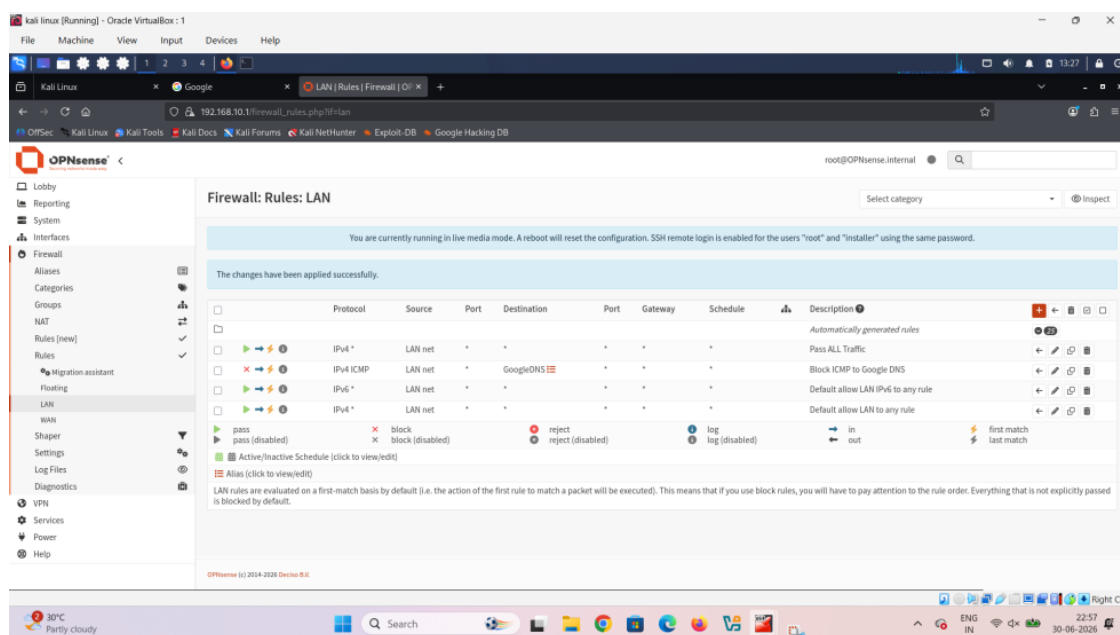


Figure 5: Firewall > Rules > LAN with “Pass ALL Traffic” positioned as the first custom rule, above “Block ICMP to Google DNS.” Because OPNsense stops at the first match, every packet from LAN net — including ICMP to the GoogleDNS alias — now matches the Pass rule first and is allowed, making the block rule beneath it unreachable.

Why this breaks the block

With “Pass ALL Traffic” evaluated first, every packet from the LAN, including pings to 8.8.8.8 and 8.8.4.4, satisfies that rule’s criteria before the firewall ever reaches the ICMP block rule below it. The block rule is not deleted or disabled — it is simply unreachable, which is a common and easy-to-miss misconfiguration.

5.2 Correction — Restoring proper rule order

The rule order was corrected by moving “Block ICMP to Google DNS” back above “Pass ALL Traffic,” restoring the intended filtering behaviour: specific block rules are evaluated before broader allow rules.

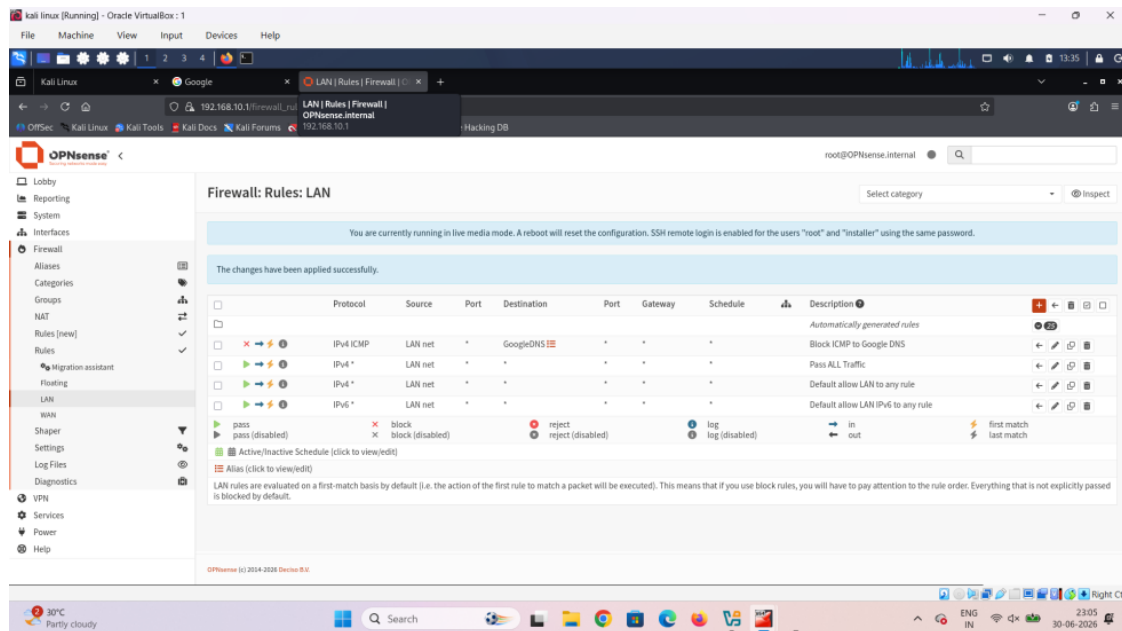


Figure 6: Firewall › Rules › LAN after correcting the order: “Block ICMP to Google DNS” is once again evaluated before “Pass ALL Traffic” and the default allow rules, restoring the intended block behaviour.

5.3 Rule-order takeaway

Rule Order	Effective Behaviour
Block rule listed above Pass/Allow rule	ICMP to GoogleDNS is dropped — rule works as intended
Pass/Allow rule listed above Block rule	ICMP to GoogleDNS is allowed — block rule is bypassed entirely

6. Step 5 — Supplementary Rule: Blocking a Specific Host

As an extension of the exercise, an additional rule was created to block all outbound internet access for a single host on the network, identified by its IP address (192.168.1.161), demonstrating that block rules can target specific machines rather than only protocol/destination combinations.

Rule parameters

Field	Value
Action	Block
Protocol	IPv4 *
Source	192.168.1.161
Destination	*
Description	Block Windows Internet

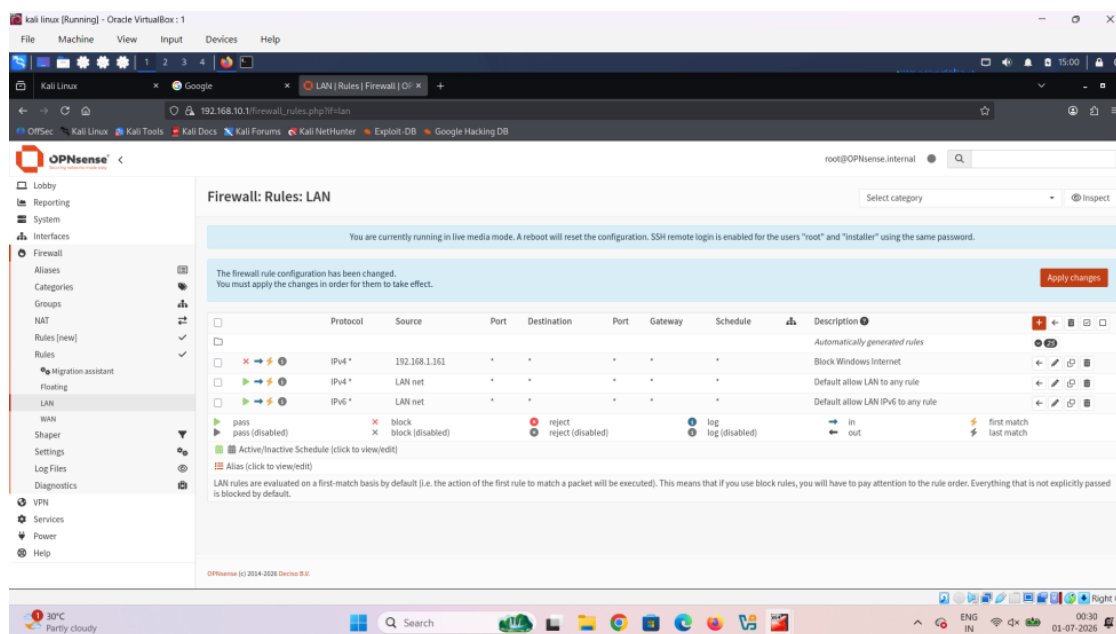


Figure 7: Firewall › Rules › LAN showing the new “Block Windows Internet” rule targeting source host 192.168.1.161, positioned above the default allow rules. The banner “The firewall rule configuration has been changed — you must apply the changes” indicates the rule was saved but not yet committed with the Apply changes button.

Important step

In OPNsense, saving a rule only stages the change. The orange “Apply changes” button must be clicked for the new rule to take effect on live traffic — a step worth double-checking after every rule edit.

7. Final Verified Ruleset

After all steps above, the LAN ruleset (excluding the supplementary host rule tested independently in Section 6) was left in its correct, working order:

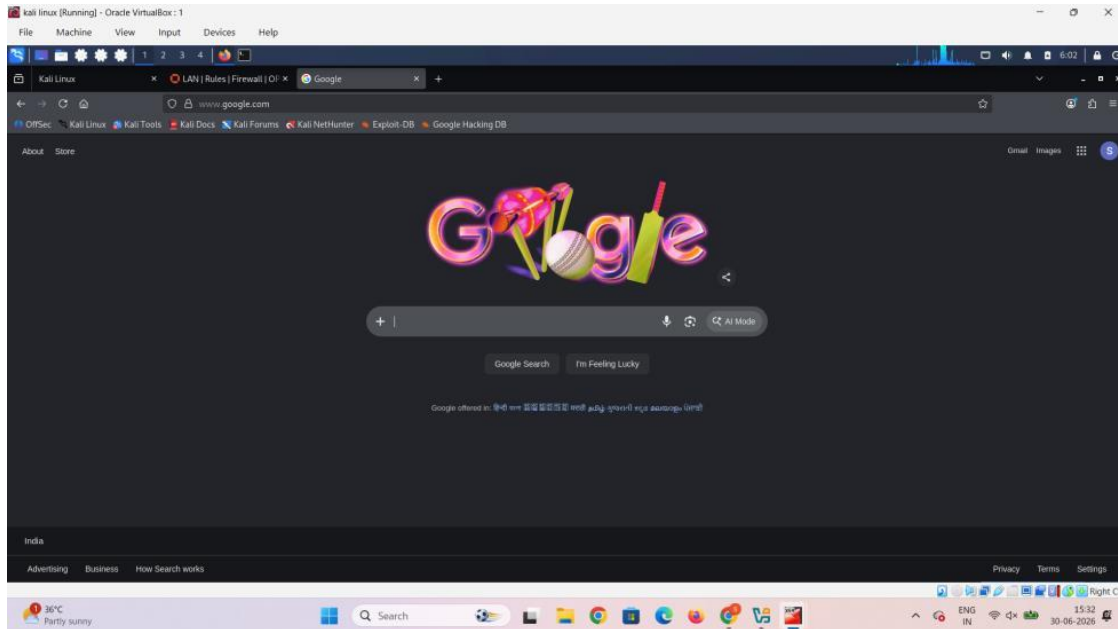


Figure 8: Baseline Firewall › Rules › LAN view confirming the “Block ICMP to Google DNS” rule (destination 8.8.8.8/32) sits above the automatically generated default allow rules, with the change-applied confirmation banner visible.

#	Action	Protocol	Source	Destination	Description
1	Block	IPv4 ICMP	LAN net	GoogleDNS	Block ICMP to Google DNS
2	Pass	IPv4 *	LAN net	*	Default allow LAN to any rule
3	Pass	IPv6 *	LAN net	*	Default allow LAN IPv6 to any rule

8. Answers to Exercise Questions

Q1. What happens when a Block rule is placed after an Allow-all rule?

The Block rule becomes unreachable. Because OPNsense evaluates rules top-to-bottom and stops at the first match, every packet is already accepted by the Allow-all rule before the firewall reaches the Block rule beneath it. This was demonstrated directly in Section 5.1, where moving “Pass ALL Traffic” above the ICMP block rule caused pings to succeed despite the block rule still being present and enabled.

Q2. How does an alias improve rule management compared to hard-coded IP addresses?

An alias lets multiple hosts (here, 8.8.8.8 and 8.8.4.4) be referenced by a single rule and a single name. Updating the alias content automatically updates every rule that references it, with no need to edit or duplicate rules. This was verified in Section 4.2, where expanding the GoogleDNS alias immediately extended the existing block rule to cover the new address without any rule changes.

Q3. Why did ping tests initially show 100% packet loss, and what confirmed the rule was the cause?

The 100% packet loss on all ping tests (Section 4) matched the expected effect of the ICMP block rule. This was corroborated by the Aliases page (Section 2), which showed the GoogleDNS alias with 66 matched packets and 66 blocked packets, directly linking the observed connectivity failure to the firewall rule rather than a network or VM misconfiguration.

Q4. What manual step is required after saving a new or edited rule?

Saving a rule only stages it. The administrator must click Apply changes for OPNsense to compile and load the new ruleset into the live packet filter, as shown by the pending-changes banner in Section 6.

Q5. Can a firewall rule target a single host instead of a whole subnet?

Yes. The supplementary rule in Section 6 used a single host address (192.168.1.161) as its Source field, showing that OPNsense rules can be scoped as narrowly as one machine or as broadly as an entire network, alias, or “any.”

9. How Firewall Rule Order Affects Traffic Filtering

OPNsense (like pfSense and most stateful packet filters derived from pf) processes the rules on an interface sequentially, from the top of the list to the bottom, for every new connection. The very first rule whose Protocol, Source, Destination, and Port criteria match the packet is the rule that is applied — its action (Pass, Block, or Reject) is executed immediately, and no further rules on that interface are checked for that packet.

This “first match wins” behaviour has three practical consequences that this lab demonstrated directly:

- Specificity must come first — the most specific rules (a single host, a single alias, a single protocol) should always sit above more general rules, so they are evaluated first. In Section 3, the ICMP block rule was placed above the default allow-all rule for exactly this reason.
- An allow rule can silently override a block rule below it — a broad Pass rule placed above a Block rule will silently swallow all matching traffic before the Block rule is ever reached. Section 5.1 showed this concretely: moving “Pass ALL Traffic” above the ICMP block rule caused ping to succeed again, even though the block rule remained enabled and unchanged.
- Default allow rules sit at the bottom for a reason — the automatically generated “Default allow LAN to any rule” exists precisely to permit everything not explicitly filtered above it. Any custom rule intended to restrict LAN traffic must be placed above this default rule, or it will never take effect — the LAN interface will simply behave as fully open.

Practical rule-writing principle

Order rules from most specific / most restrictive at the top to most general / most permissive at the bottom. When troubleshooting a rule that “doesn't seem to work,” check its position in the list before assuming the rule itself is misconfigured — as this lab showed, the rule can be perfectly correct and still be bypassed purely due to ordering.

10. Conclusion

This lab successfully demonstrated the end-to-end process of configuring OPNsense firewall rules: grouping destination hosts into a reusable alias, writing a Block rule referencing that alias, validating the rule's effect through live ping tests from a Kali Linux client, and diagnosing a rule-order misconfiguration that temporarily bypassed the block. A supplementary host-based rule further confirmed that rules can be scoped to individual machines just as easily as to protocols or aliases.

The key operational lesson from this exercise is that a firewall ruleset is not just a set of independent rules but an ordered sequence evaluated first-match-wins — correct rule placement is as important as correct rule content. All findings in this report are supported by firewall configuration screenshots and terminal-based connectivity evidence captured during the exercise.

Result

All objectives met — alias created, block rule verified via 100% ICMP packet loss, rule-order behaviour reproduced and corrected, and a supplementary host-block rule staged successfully.